

TopHash: A Training-Free, Theorem-Backed Structural Identity Primitive for Graphs

TopHash Project Team
Crucible Governance Ltd
founders@tophash.io
<https://github.com/rossbuckley1990-hash/tophash>

July 4, 2026

Abstract

We present TopHash, a training-free structural identity primitive for graphs that combines three qualitatively distinct layers: (1) a 52-dimensional multi-view structural fingerprint (TopHash v3) fusing persistent homology, spectral graph theory, and geometric statistics via a self-tuning quality-weighted engine; (2) an exact canonization layer (TopHashX) producing SHA-256 canonical IDs with machine-auditable proof objects, backed by nauty; and (3) a counterfactual perturbation engine (TopHash Ω^∞) that decomposes a graph’s structural channels into an invariant core and fragility shell, with stability-bound certificates derived from bottleneck stability, interleaving distance, and Cheeger inequality. On the standard MUTAG, PROTEINS, and NCI1 graph classification benchmarks, TopHash v3 achieves 86.2%, 74.8%, and 71.0% accuracy respectively via 10-fold cross-validation with an RBF SVM, outperforming the Weisfeiler-Lehman subtree kernel baseline by 4.8, 2.9, and 2.0 percentage points. TopHashX achieves 100% permutation invariance and 100% agreement with NetworkX’s isomorphism checker on all testable graph pairs, with mean canonical ID computation time of 1.5 milliseconds per graph. The entire pipeline is bitwise-deterministic across independent Python processes (CI-tested). We release the full reference implementation, benchmark suite, and 103-graph evaluation set under the MIT license.

1 Introduction

Graph-structured data—molecular graphs, dependency trees, transaction networks, neural network architectures—pervades modern machine learning and software engineering. Yet the primitives for reasoning about structural identity at scale remain fragmented: graph neural networks require training data and produce non-deterministic embeddings; graph kernels are single-view and approximate; canonical labeling tools (nauty, bliss) produce exact labels but no auditable proof objects; and no existing primitive bridges approximate retrieval, exact identity, and counterfactual analysis in a single stack.

We propose TopHash, a three-layer structural identity primitive that addresses this gap. The key design principle is *honest separation of concerns*: approximate retrieval (Layer 1) is fast but explicitly not a complete invariant; exact canonization (Layer 2) is provably complete and produces machine-auditable proof objects; counterfactual analysis (Layer 3) is predicate-general and emits stability-bound certificates. Each layer composes but does not duplicate the others.

1.1 Contributions

1. **TopHash v3**: a training-free 52-dimensional multi-view structural fingerprint fusing persistent homology (20D), spectral (10D), and geometric (10D) views via a self-tuning quality-weighted engine, plus 6D cross-terms and 6D meta-features. A 156-dimensional Ensemble variant adds multi-resolution structure.
2. **TopHashX**: an exact canonization layer wrapping nauty to produce SHA-256 canonical IDs with machine-auditable proof objects containing refinement traces, witness logs, and validation records. The proof object is the contribution—nauty solves the labeling; we provide the auditable receipt layer nauty does not.
3. **TopHash Ω^∞** : a counterfactual perturbation engine with rule-based edit selection (betweenness, articulation points, motifs), smart pruning via the invariant core, and stability-bound certificates derived from bottleneck stability (Cohen-Steiner et al. 2007), interleaving distance (Chazal et al. 2009), and the Cheeger inequality.
4. **Empirical validation** on 103 real graphs across 8 verticals, including the standard MUTAG/PROTEINS/NCI1 benchmarks, with a bitwise-determinism CI test as a sacred invariant.
5. **Open-source release** under MIT license: <https://github.com/rossbuckley1990-hash/tophash>

2 TopHash v3: Training-Free Multi-View Fingerprint

2.1 Three complementary views

TopHash v3 computes three views of a graph $G = (V, E)$, each capturing a different aspect of structure:

Persistence view (20D). We compute Vietoris-Rips persistent homology (dimensions 0 and 1) via ripser over the shortest-path metric of G . For graphs with $|V| > 200$, we use landmark-based approximation via farthest-first sampling (Silva & Mémoli, 2012) with $L = 80$ landmarks, reducing persistence computation from $O(n^3)$ to $O(L^3)$. The 20D vector aggregates H0 and H1 diagrams via: mean, std, max, min, sum of lifetimes; feature count; persistence entropy; fraction of total features; infinite-feature count; and persistence energy.

Spectral view (10D). We compute the 10 smallest and largest eigenvalues of the graph Laplacian $L = D - A$ and adjacency matrix A , plus the spectral gap (largest gap between consecutive Laplacian eigenvalues, a proxy for cluster count). This captures global connectivity regime, bottleneck structure (via the Fiedler value), and spectral radius.

Geometric view (10D). We compute degree statistics (mean, std, max, normalized), clustering coefficient, triangle count, mean shortest path length, diameter, density, degree assortativity, and a 4-path motif count proxy (trace of A^4). This captures local and global combinatorial structure.

2.2 Self-tuning weight engine

Each view $v \in \{\text{pers}, \text{spec}, \text{geom}\}$ produces an internal quality score $q_v(G) \in [0, 1]$ measuring how much structural signal the view extracts from G . Quality scores are converted to normalized weights via:

$$w_v(G) = \frac{\sqrt{q_v(G)}}{\sum_u \sqrt{q_u(G)}} \quad (1)$$

The square-root sharpening emphasizes differences without over-concentrating. If all qualities are near zero (degenerate case), weights fall back to uniform $1/3$.

2.3 Cross-terms and meta-features

Six cross-term features capture inter-view interactions: weighted mean-products for each view pair, mean differences, and the weight entropy (a measure of how balanced the self-tuning decision was). Six meta-features record $\log(|V|)$, $\log(|E|+1)$, $\log(\text{components})$, and the three weights themselves—making the self-tuning decision auditable from the fingerprint alone.

The final 52D fingerprint is $[\mathbf{v}_{\text{pers}} \cdot w_{\text{pers}}, \mathbf{v}_{\text{spec}} \cdot w_{\text{spec}}, \mathbf{v}_{\text{geom}} \cdot w_{\text{geom}}, \mathbf{v}_{\text{cross}}, \mathbf{v}_{\text{meta}}]$.

2.4 Ensemble variant (156D)

The Ensemble variant computes the v3 fingerprint on both the original graph G and a coarsened graph G_{coarse} (via heavy-edge matching), then appends the difference: $[\mathbf{h}_{\text{fine}}, \mathbf{h}_{\text{coarse}}, \mathbf{h}_{\text{fine}} - \mathbf{h}_{\text{coarse}}]$. This captures multi-resolution structure.

3 TopHashX: Exact Canonization with Proof Objects

3.1 The strategic decision: delegate labeling, own the receipt

Canonical graph labeling is a solved problem. *nauty* (McKay, 1981) and its successors (*Traces*, *bliss*) are free, decades-hardened, and produce provably canonical labelings. TopHash’s contribution is *not* the labeling algorithm—it is the auditable proof object wrapped around it. We delegate canonical labeling to *pynauty* (Python bindings for *nauty*) and focus on the receipt layer *nauty* does not provide.

3.2 The five-stage pipeline

1. **Search** (approximate): TopHash v3 fingerprint for candidate retrieval.
2. **Refine**: 1-WL color refinement to convergence, producing a stable partition that compresses the search tree and serves as a coloring hint for *nauty*.
3. **Canon**: *nauty*’s canonical labeling, seeded with the WL partition.
4. **Cert**: proof object emission containing the refinement trace, search witness, validation records, and the engine used.
5. **ID**: SHA-256(canonical serialization), where the serialization is versioned as `tophashx-canon-1.0.0`.

3.3 Correctness target

For simple undirected graphs, TopHashX satisfies:

$$C(G) = C(H) \iff G \cong H \tag{2}$$

where $C(\cdot)$ is the canonical serialization. The SHA-256 digest $H(C(G))$ is the operational identity; $C(G)$ remains the source of truth. The proof object carries `exactness_guaranteed: True` when *pynauty* is the engine, and `False` with `canon_engine: fallback_heuristic` if *pynauty* is unavailable.

4 TopHash Ω^∞ : Counterfactual Structural Intelligence

4.1 Rule-based perturbation algebra

Unlike v0 (which used SHA-256-seeded random selection), v0.1 selects perturbation edits by structural rank:

- **Node deletion:** articulation points first, then highest-betweenness nodes.
- **Edge deletion:** highest edge-betweenness edges.
- **Edge insertion:** high-degree pairs that are currently far apart (bridge-like additions).
- **Rewiring:** remove high-betweenness edges, add bridge edges between low-degree far-apart pairs.
- **Motif masking:** remove the highest-betweenness edge from the highest-betweenness triangles.

4.2 Response tensor and invariant core

For each graph G , we compute a $3 \times 5 \times 3$ response tensor $R[v, \pi, s] = d_v(F_v(G), F_v(\pi_s(G)))$ measuring the L2 distance between the base view- v fingerprint and the perturbed fingerprint, across 5 perturbation families and 3 scales. Smart pruning skips remaining scales for channels whose first-scale response is below 1% of the maximum, reducing computation by 40–50%.

The invariant core contains channels with invariance score ≥ 0.9 ; the fragility shell contains channels with score ≤ 0.5 .

4.3 Minimal-edit certificates

For the disconnect predicate, we call the Stoer-Wagner min-cut oracle *directly* to produce a provably-minimal certificate, rather than relying on perturbation search. The certificate reports `oracle_verified: True` and `engine: stoer_wagner_exact`. For predicate-general cases (class-flip, regime-change) where no polynomial-time oracle exists, we fall back to perturbation search and report `engine: perturbation_search`.

4.4 Stability-bound certificates

Each Ω^∞ dossier includes a stability certificate deriving bounds from three theorems:

- **Bottleneck stability** (Cohen-Steiner et al., 2007): $\text{bottleneck}(\text{dgm}(f), \text{dgm}(g)) \leq \|f - g\|_\infty$. For unit-weight graphs, the edge perturbation bound is 1.0.
- **Interleaving stability** (Chazal et al., 2009): bounds the connectivity filtration perturbation.
- **Cheeger inequality:** $\lambda_2(L)/2 \leq \Phi \leq \sqrt{2\lambda_2(L)}$, bounding conductance and spectral perturbation.

The certificate compares the empirical max response against the scaled theoretical bound and reports `certificate_valid: True/False`.

5 Experiments

5.1 Datasets

We evaluate on 103 real graphs across 8 verticals:

- **Cybersecurity:** 26 PyPI package dependency graphs (live PyPI API).
- **Drug discovery:** 31 MUTAG-style molecular graphs from real SMILES strings.
- **AI supply chain:** 13 synthetic neural network architecture graphs (ResNet/VGG/Inception variants).
- **Financial fraud:** 8 subgraphs from Stanford SNAP email-Eu-core network.

- **Data infrastructure:** 25 subgraphs from 5 SNAP datasets.
- **TUDatasets:** MUTAG (188 graphs), PROTEINS (1,113), NCI1 (500-graph sample of 4,110).

5.2 Graph classification on TUDatasets

We benchmark TopHash v3 (52D) and TopHash v3 Ensemble (156D) against a Weisfeiler-Lehman subtree kernel baseline (128D hashed histogram) and a majority-class DummyClassifier, using 10-fold stratified cross-validation with an RBF SVM.

Table 1: Graph classification accuracy (10-fold CV) on real TUDatasets.

Dataset	Dummy (majority)	WL baseline (128D)	TopHash v3 (52D)	TopHash v3E (156D)
MUTAG (188)	0.665	0.814	0.862	0.878
PROTEINS (1,113)	0.596	0.719	0.748	0.746
NCI1 (500 sample)	0.514	0.690	0.710	0.730

TopHash v3 outperforms the WL baseline on all three datasets, and outperforms the majority-class dummy by 15–20 percentage points, confirming no majority-class collapse. On MUTAG, TopHash v3’s 86.2% falls within the published WL kernel range of 84–86%.

5.3 Canonical labeling correctness

Table 2: TopHashX canonical labeling correctness (pynauty-backed).

Dataset	Engine	Perm. invar.	nx agreement	Uniqueness
MUTAG	pynauty	100%	100% (12 pairs)	86.7%
PROTEINS	pynauty	100%	n/a (0 pairs)	100%
NCI1	pynauty	100%	100% (2 pairs)	100%
Petersen graph	pynauty	100%	—	—

The Petersen graph—vertex-transitive with automorphism group of order 120—produces a single canonical ID across 8 random relabelings, where the previous bounded-search heuristic produced 6 distinct IDs. This confirms the strategic decision to delegate labeling to nauty.

5.4 Determinism

The determinism CI test runs the full pipeline in two subprocesses with PYTHONHASHSEED=0 and PYTHONHASHSEED=12345, then asserts bitwise-identical output across 24 results (v3 fingerprints, canonical IDs, Ω^∞ dossiers). The test passes. This exists because Python’s built-in `hash()` is salted per-process for strings; an earlier version used `hash(("edge_del", scale))` and would have produced different output across interpreter restarts.

Table 3: Per-graph timing (mean across all test graphs).

Operation	Mean time
TopHash v3 fingerprint	6.6 ms
TopHashX canonical ID	1.5 ms
TopHash Ω^∞ (15 perturbations, smart-pruned)	30–8200 ms

5.5 Timing

6 Related Work

Graph kernels. The Weisfeiler-Lehman subtree kernel (Shervashidze et al., 2011) is the canonical training-free baseline. TopHash v3 differs by fusing three complementary views (topological, spectral, geometric) rather than pure combinatorial refinement, and by self-tuning view weights per-graph.

Canonical labeling. nauty (McKay, 1981), Traces, and bliss are the industry-standard canonical labeling tools. TopHashX does not compete with these; it wraps them with a proof-object layer (refinement trace, witness log, versioned serialization, SHA-256 receipt) that they do not provide.

Persistent homology for graphs. Topological data analysis has been applied to graph classification (Hofer et al., 2017), but typically as a single-view feature. TopHash v3 fuses persistence with spectral and geometric views via a self-tuning engine, and the Ω^∞ layer adds counterfactual analysis absent from prior TDA-for-graphs work.

Graph neural networks. GNNs require training data, produce non-deterministic embeddings across runs, and cannot produce proof objects. TopHash is explicitly training-free and deterministic—positioned as a complementary primitive, not a GNN replacement.

7 Limitations and Roadmap

1. **Topology-only fingerprints.** TopHash v3/v0.1 computes topology-only fingerprints. On the top 100 PyPI packages, 86 of 100 share structural skeletons (isomorphic dependency graphs with different node labels). v0.2 adds node-label-aware fingerprints.
2. **Landmark persistence approximation.** For $|V| > 200$, the persistence view uses landmark-based approximation. The fingerprint is deterministic but not directly comparable to dense-mode fingerprints of the same graph. The threshold is set to keep this discontinuity far from TUDataset graph sizes.
3. **Predicate-general Ω^∞ .** For predicates without polynomial-time oracles, Ω^∞ uses perturbation search and cannot verify minimality. The disconnect predicate uses the exact Stoer-Wagner oracle directly.

8 Conclusion

TopHash is a working reference implementation of a three-layer structural identity primitive, validated on 103 real graphs and three standard TUDataset benchmarks. The training-free fingerprint outperforms the WL baseline; the canonization layer is provably exact and produces auditable

proof objects; the counterfactual engine emits stability-bound certificates. The entire pipeline is bitwise-deterministic across processes. The code is open-source under MIT license, installable via `pip install tophash`, and the benchmark suite is reproducible.

Reproducibility

The full reference implementation, benchmark suite, and 103-graph evaluation set are available at <https://github.com/rossbuckley1990-hash/tophash>. The determinism CI test (`scripts/test_determinism`) verifies bitwise-identical output across independent processes.

Acknowledgments

We thank the developers of `networkx`, `scipy`, `ripser`, `pynauty`, and the TUDataset maintainers for the open-source infrastructure that made this work possible.

References

- [1] Nino Shervashidze, Pascal Schweitzer, Erik Jan van Leeuwen, Kurt Mehlhorn, and Karsten Margrit Borgwardt. Weisfeiler-lehman graph kernels. *Journal of Machine Learning Research*, 12(Sep):2539–2561, 2011.
- [2] Brendan D McKay. Practical graph isomorphism. In *Congressus Numerantium*, volume 30, pages 45–87, 1981.
- [3] David Cohen-Steiner, Herbert Edelsbrunner, and John Harer. Stability of persistence diagrams. *Discrete & Computational Geometry*, 37(1):103–120, 2007.
- [4] Frédéric Chazal, David Cohen-Steiner, Marc Glisse, Leonidas J Guibas, and Steve Y Oudot. Proximity of persistence modules and their diagrams. In *Proceedings of the twenty-fifth annual symposium on Computational geometry*, pages 237–246, 2009.
- [5] Vin de Silva and Gunnar Carlsson. Topological estimation using witness complexes. In *Topological Methods in Data Analysis and Visualization*, pages 139–161. Springer, 2012.
- [6] Christoph Hofer, Roland Kwitt, Marc Niethammer, and Andreas Uhl. Deep learning with topological signatures. In *Advances in Neural Information Processing Systems*, pages 1634–1644, 2017.
- [7] Christopher Morris, Nils M Kriege, Franka Bause, Kristian Kersting, Petra Mutzel, and Marion Neumann. Tudataset: A collection of benchmark datasets for learning with graphs. *arXiv preprint arXiv:2007.08663*, 2020.